

Software Quality 2025/26

Project 2

Eduardo Ervideira (72420)
Noah Andriamampianina (76197)

Maio 2026

GitHub repository: <https://github.com/eduardoervideira/Bukkit>

1 Introduction

This report documents the application of black-box and white-box testing techniques to Bukkit, which is an open source Java API and plugin framework for Minecraft multiplayer servers.

Two functions were selected for testing:

- `Color.fromRGB(int red, int green, int blue)`: Validates and creates a `Color` object from RGB values (0-255).
- `Note(int octave, Tone tone, boolean sharped)`: Validates and creates a musical note based on octave, tone, and sharpness.

The report is structured as follows:

- Section 2 covers `Color.fromRGB` with Equivalence Class Testing, Boundary Value Analysis, and Control Flow-Based Testing.
- Section 3 covers `Note` with Cause-Effect Graphing, Category Partition Testing, and White-box Testing (Control Flow and Data Flow).

2 Function: `Color.fromRGB`

2.1 Source Code

The `Color.fromRGB` function and its private constructor are shown below. The full implementation is available at: `Color.java`.

Listing 1: `Color.fromRGB` Implementation

```
private Color(int red, int green, int blue) {
    Validate.isTrue(red >= 0 && red <= BIT_MASK, "Red is not between 0-255: ", red);
    Validate.isTrue(green >= 0 && green <= BIT_MASK, "Green is not between 0-255: ", green);
    Validate.isTrue(blue >= 0 && blue <= BIT_MASK, "Blue is not between 0-255: ", blue);

    this.red = (byte) red;
    this.green = (byte) green;
    this.blue = (byte) blue;
}
```

```

public static Color fromRGB(int red, int green, int blue) throws
    IllegalArgumentException {
    return new Color(red, green, blue);
}

```

2.2 Equivalence Class Testing

For rationale, the function validates three parameters (**red**, **green**, **blue**), each with a valid domain of $[0, 255]$. Equivalence Class Testing partitions the input into valid and invalid classes for each parameter.

2.2.1 Equivalence Classes

Table 1: Equivalence Classes for `Color.fromRGB`

Parameter	Invalid (Lower)	Valid	Invalid (Upper)
red	$\text{red} < 0$	$0 \leq \text{red} \leq 255$	$\text{red} > 255$
green	$\text{green} < 0$	$0 \leq \text{green} \leq 255$	$\text{green} > 255$
blue	$\text{blue} < 0$	$0 \leq \text{blue} \leq 255$	$\text{blue} > 255$

Each parameter has 3 equivalence classes, therefore $|\text{red}| = |\text{green}| = |\text{blue}| = 3$.

2.2.2 Test Cases

Weak Equivalence: The number of test cases equals the maximum number of classes across all parameters: $\max(|\text{red}|, |\text{green}|, |\text{blue}|) = 3$ tests.

Strong Equivalence: The cartesian product of partition subsets, therefore $|\text{red}| * |\text{green}| * |\text{blue}| = 27$ tests.

Table 2: Weak Equivalence Test Cases

Test ID	Red	Green	Blue	Classes Covered	Expected Outcome
WE1	-1	-1	-1	Invalid Lower (R, G, B)	<code>IllegalArgumentException</code>
WE2	128	128	128	Valid (R, G, B)	Color object created
WE3	256	256	256	Invalid Upper (R, G, B)	<code>IllegalArgumentException</code>

Table 3: Strong Equivalence Test Cases

Test ID	Red	Green	Blue	Expected Outcome
SE1	-1	-1	-1	IllegalArgumentException
SE2	-1	-1	128	IllegalArgumentException
SE3	-1	-1	256	IllegalArgumentException
SE4	-1	128	-1	IllegalArgumentException
SE5	-1	128	128	IllegalArgumentException
SE6	-1	128	256	IllegalArgumentException
SE7	-1	256	-1	IllegalArgumentException
SE8	-1	256	128	IllegalArgumentException
SE9	-1	256	256	IllegalArgumentException
SE10	128	-1	-1	IllegalArgumentException
SE11	128	-1	128	IllegalArgumentException
SE12	128	-1	256	IllegalArgumentException
SE13	128	128	-1	IllegalArgumentException
SE14	128	128	128	Color object created
SE15	128	128	256	IllegalArgumentException
SE16	128	256	-1	IllegalArgumentException
SE17	128	256	128	IllegalArgumentException
SE18	128	256	256	IllegalArgumentException
SE19	256	-1	-1	IllegalArgumentException
SE20	256	-1	128	IllegalArgumentException
SE21	256	-1	256	IllegalArgumentException
SE22	256	128	-1	IllegalArgumentException
SE23	256	128	128	IllegalArgumentException
SE24	256	128	256	IllegalArgumentException
SE25	256	256	-1	IllegalArgumentException
SE26	256	256	128	IllegalArgumentException
SE27	256	256	256	IllegalArgumentException

2.3 Boundary Value Analysis (BVA)

Boundary Value Analysis focuses on testing values at and around the boundaries of each input domain, where defects are statistically more likely to occur. The boundary values considered are:

- **Normal variants:** $\{0, 1, 128, 254, 255\}$ representing *min*, *min+*, *nom*, *max-*, and *max*.
- **Robust variants:** additionally include $\{-1, 256\}$ representing *min-* and *max+* (out-of-range values).

2.3.1 Number of Test Cases

Table 4: Number of Test Cases for BVA Variants ($n = 3$)

Variant	Formula	Total
Simple BVA	$4n + 1$	13
Robust BVA	$6n + 1$	19
Worst Case BVA	5^n	125
Robust Worst Case BVA	7^n	343

Rationale

- **Simple BVA** ($4n + 1 = 13$): assumes the single-fault hypothesis. For each variable, the 5 boundary values are tested while keeping the remaining variables at their nominal value. The nominal-only case is counted once instead of n times.
- **Robust BVA** ($6n + 1 = 19$): extends Simple BVA by adding the two out-of-range values per variable, allowing explicit verification of the exception-throwing behavior.
- **Worst Case BVA** ($5^n = 125$): drops the single-fault hypothesis and considers all combinations of boundary values across the three parameters (Cartesian product).
- **Robust Worst Case BVA** ($7^n = 343$): the most exhaustive variant, combining the multi-fault assumption with the robust value set.

2.3.2 Simple BVA Test Battery

Table 5: Simple BVA Test Cases for `Color.fromRGB`

Test ID	Red	Green	Blue	Expected Outcome
SBVA1	0	128	128	Color object created
SBVA2	1	128	128	Color object created
SBVA3	254	128	128	Color object created
SBVA4	255	128	128	Color object created
SBVA5	128	0	128	Color object created
SBVA6	128	1	128	Color object created
SBVA7	128	254	128	Color object created
SBVA8	128	255	128	Color object created
SBVA9	128	128	0	Color object created
SBVA10	128	128	1	Color object created
SBVA11	128	128	254	Color object created
SBVA12	128	128	255	Color object created
SBVA13	128	128	128	Color object created

2.3.3 Robust BVA Test Battery

Table 6: Robust BVA Test Cases for `Color.fromRGB`

Test ID	Red	Green	Blue	Expected Outcome
RBVA1	-1	128	128	<code>IllegalArgumentException</code>
RBVA2	0	128	128	<code>Color</code> object created
RBVA3	1	128	128	<code>Color</code> object created
RBVA4	254	128	128	<code>Color</code> object created
RBVA5	255	128	128	<code>Color</code> object created
RBVA6	256	128	128	<code>IllegalArgumentException</code>
RBVA7	128	-1	128	<code>IllegalArgumentException</code>
RBVA8	128	0	128	<code>Color</code> object created
RBVA9	128	1	128	<code>Color</code> object created
RBVA10	128	254	128	<code>Color</code> object created
RBVA11	128	255	128	<code>Color</code> object created
RBVA12	128	256	128	<code>IllegalArgumentException</code>
RBVA13	128	128	-1	<code>IllegalArgumentException</code>
RBVA14	128	128	0	<code>Color</code> object created
RBVA15	128	128	1	<code>Color</code> object created
RBVA16	128	128	254	<code>Color</code> object created
RBVA17	128	128	255	<code>Color</code> object created
RBVA18	128	128	256	<code>IllegalArgumentException</code>
RBVA19	128	128	128	<code>Color</code> object created

2.3.4 Worst Case BVA Test Battery

The Worst Case BVA test battery consists of all $5^3 = 125$ combinations obtained from the Cartesian product of the boundary value set $\{0, 1, 128, 254, 255\}$ for the three parameters. Since all values lie within the valid domain $[0, 255]$, *all 125 test cases are expected to successfully create a `Color` object*. This evidences a known limitation of this technique: without including invalid values, Worst Case BVA does not exercise the exception-throwing behavior of the function.

Table 7: Worst Case BVA Test Cases for `Color.fromRGB` (representative subset)

Test ID	Red	Green	Blue	Expected Outcome
WCBVA1	0	0	0	<code>Color</code> object created
WCBVA2	0	0	1	<code>Color</code> object created
WCBVA3	0	0	128	<code>Color</code> object created
WCBVA4	0	0	254	<code>Color</code> object created
WCBVA5	0	0	255	<code>Color</code> object created
WCBVA6	0	1	0	<code>Color</code> object created
⋮	⋮	⋮	⋮	⋮
WCBVA63	128	128	128	<code>Color</code> object created
⋮	⋮	⋮	⋮	⋮
WCBVA125	255	255	255	<code>Color</code> object created

2.3.5 Robust Worst Case BVA Test Battery

The Robust Worst Case BVA test battery consists of all $7^3 = 343$ combinations obtained from the Cartesian product of the robust boundary value set $\{-1, 0, 1, 128, 254, 255, 256\}$ for the three parameters. A test case produces a valid `Color` object only if all three parameters are within $[0, 255]$; otherwise, an `IllegalArgumentException` is thrown. This yields $5^3 = 125$ successful cases and $343 - 125 = 218$ exception cases.

Table 8: Robust Worst Case BVA Test Cases for `Color.fromRGB` (representative subset)

Test ID	Red	Green	Blue	Expected Outcome
RWCBVA1	-1	-1	-1	<code>IllegalArgumentException</code>
RWCBVA2	-1	-1	0	<code>IllegalArgumentException</code>
RWCBVA3	-1	-1	1	<code>IllegalArgumentException</code>
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
RWCBVA172	128	128	128	Color object created
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
RWCBVA343	256	256	256	<code>IllegalArgumentException</code>

2.4 Used Prompt

```
I have a Java method Color.fromRGB(int red, int green, int blue) that
validates
each parameter to be in [0, 255] and throws IllegalArgumentException
otherwise.
Generate JUnit 5 tests from these specifications:

Simple BVA (13 tests, SBVA1-SBVA13): vary one parameter at a time across
{0, 1,
128, 254, 255}, keep others at 128.
Robust BVA (19 tests): Simple BVA + out-of-range values -1 and 256 per
parameter.
Worst Case BVA (125 tests): Cartesian product of {0, 1, 128, 254, 255}.
Robust Worst Case BVA (343 tests): Cartesian product of {-1, 0, 1, 128,
254,
255, 256}.

Use @ParameterizedTest with @CsvSource for the smaller batteries and
@MethodSource with programmatic generation for the larger ones. Each test
asserts either successful Color creation (all parameters in [0, 255]) or
IllegalArgumentException.
```

2.5 Control Flow-Based Testing

For rationale, the function has three validation checks (`Validate.isTrue`) that throw exceptions if any condition fails. Control Flow-Based Testing ensures all statements, decisions, and paths are covered.

TODO

3 Function: Note

Placeholder for future content.

4 Conclusion

Placeholder for future content.